Nyaya Saathi: a grounded AI legal helper for India

Kuku AI GeneralistAssignment | Problem2 | Name:_Irlapati Manas Chandra_____ | Rollno:__23045062_____ | Branch:___Chemical_____

1. Why this problem, and scope

I chose Problem 2 because it can be tested end to end with my own cases, and its main risk (an AI inventing laws) can be designed against and measured. **Covers:** four everyday problems where people give up: rent-deposit refusal, consumer refund not received, unpaid salary, bounced cheque. **Does not cover:** criminal, family, property and tax matters; these trigger an immediate 'see a lawyer / free legal aid' response instead of advice.

2. Assumptions (brief was silent)

(a) Users write in Hindi/Hinglish, English or a regional language, mostly on phones; (b) the product gives legal information and preparation, never representation or guaranteed outcomes; (c) a small hand-checked law list beats a model's memory for a regulated-feeling domain; (d) voice input is out of scope for the 2-3 hour build (a transcription step in front of the same pipeline would add it); (e) state-specific rules (e.g. rent law) cannot be answered precisely without the user's state, so the tool asks.

3. How it works

Pipeline per message: (1) rule-based **safety flags** (arrest, violence, court notice, property/divorce) fire in plain code, before and independent of the model; (2) keyword **retrieval** picks entries from a hand-written **legal notes list** (7 entries: consumer-complaint limit and forums, 3-year money-recovery limit, state rent law, s.138 cheque deadlines, wages forum, free legal aid); (3) the model receives only those notes and must **cite by id** (e.g. [CPA-69]); anything else must be labelled 'general understanding, please verify'; it must say 'I cannot verify' about sections not in the notes; (4) three outputs: guided chat that asks at most 2 questions at a time, a **case summary** (what happened, proof held vs missing, marked 'not provided' where unknown), a **draft legal notice** with [PLACEHOLDERS], and a **practise mode** where the AI plays the opposing party then gives feedback.

AI vs code vs human: AI does language, empathy, structuring a messy story and drafting. Code does safety triggers and which laws are allowed in. A human (lawyer) owns the law list and reviews it; users are told to have a lawyer review any notice. **Model is swappable** through three environment variables (provider, model name, base URL), so a stronger model can be plugged in without code changes.

4. How I avoid invented law

The model is never asked to recall statutes. It may state a section, deadline or forum only if it is in the notes it was handed, and every such claim carries an id a reviewer can check. Test 7 is a trap (a non-existent 'Section 420A'); the pass condition is a refusal to confirm it. Limitation of this design: it protects against invention only inside the four covered topics, and the notes are my own summaries which **must be verified against indiacode.nic.in** before real use (the wages statute status is explicitly marked 'verify').

5. How I know it works: real runs and honest failures

I wrote 11 test cases (Hinglish, English, Tamil, Bengali; routine, urgent and trap cases) with expected outcomes, and ran the deterministic layer (retrieval + safety flags) before and after fixes. Raw files are in **raw_outputs/**.

Run	Result	What failed
run1_before_fix	7/11	Case 6 'maarte/dhamki' (Hinglish violence) raised no danger warning. Case 8 'Will I definitely win' wrongly triggered the property/complex-matter warning because the bare word 'will' was a keyword. Cases 10, 11 (Tamil, Bengali): retrieval returned nothing, so the model would have had no legal notes.
run2_after_fix	11/11	Added Hindi-script and Romanised-Hindi violence terms, narrowed the 'will' pattern to 'will deed/dispute', added Tamil/Bengali/Hindi keywords to the notes.

Still failing / not yet proven: keyword matching is brittle (a story told without any listed word gets no notes; case 9 also pulls an unrelated consumer note); regional-language coverage is only a few keywords per language, not real understanding; the safety-flag regexes will both miss phrasings and over-fire. **Model-reply quality (language match, no invented law, correct escalation) is scored manually with the rubric in test_cases.md using run_tests.py --llm; results: [FILL IN after your run: n/11 pass and the failures you saw].** The scores above are for the code layer only and do not show the model's replies are good.

6. Scale and cost

Per conversation the cost is a few thousand input tokens (the prompt plus at most 4 notes) and a few hundred output tokens, so a low-cost model fits the free-tier brief. To scale to many problem types and states: (1) store notes as versioned records with source URL, state and last-verified date; (2) replace keyword retrieval with embeddings plus a state field collected up front; (3) a lawyer-reviewed notes pipeline and a monthly re-check; (4) log escalations and thumbs-down to grow the test set; (5) run the same tests on every model swap.

7. Tools used, and where they got it wrong

Tools: Claude (drafting the code, notes and tests), Python/Streamlit. **Where AI got it wrong:** the first draft's safety regex used the bare word 'will' (meant for wills/inheritance), which wrongly warned a user asking 'Will I win?'; it also assumed English keywords only, which my own Tamil/Bengali tests exposed. I also do not trust its recall of statute details, which is why those are in a list to verify rather than left to the model. **Next with more time:** state-aware rent rules, voice input, and a lawyer review of every note.